Cairo University
Faculty of Engineering
Computer Engineering Department

Assignment 2 Report

Full Name	Student ID
Hossam Elden Mohamed Ahmed Mohamed	9230331

Cairo University Faculty of Engineering

Electronics & Communication Department | ELC325B

3rd Year 2025-2026

Assignment 2 Report

Part I Theoretical Derivation

1.1 System Description

The system transmits binary data over an ideal (AWGN-only) channel. The pulse shape is:

$$g(t) = \sqrt{3} \cdot t, \quad 0 \le t \le T = 1\ s$$

Bit '1' is sent as $g(t)$ and bit '0' as zero. The channel adds white Gaussian noise $w(t)$ with two-sided power spectral density $N_0/2$. The receive filter output is sampled at t = T and decoded with an optimal threshold.

1.2 Average Symbol Energy

Since '0' is sent with zero energy and '1' is sent with equal probability:

$$E = (1/2) \cdot \int_0^T g^2(t)\, dt = (1/2) \cdot \int_0^1 3t^2\, dt = (1/2) \cdot [t^3]_0^1 = 1/2\ J$$

However, the *energy of the pulse itself* (which enters the matched filter design) is:

$$\|g\|^2 = \int_0^1 3t^2\, dt = 1\ J$$

All three BER formulas below use $E = \|g\|^2 = 1$ J as the signal energy parameter, consistent with E/N_0 in the plot axes.

1.3 Optimal Decision Threshold

For on-off keying with equal priors P('0') = P('1') = 1/2, the optimal threshold is the midpoint of the two possible noiseless output values (0 and the signal component). In all three cases the threshold is therefore:

$$\gamma = (\text{signal component at } T) / 2$$

Case (a) Matched Filter (Unit Energy)

The matched filter for $g(t)$ is:

$$h_a(t) = g(T - t) / \sqrt{\|g\|^2} = \sqrt{3}\cdot(T - t), \quad 0 \le t \le T$$

The filter output at t = T is the cross-correlation of the received signal with g(t):

$$y_a(T) = \int_0^T r(\tau) \cdot g(\tau)/\sqrt{E}\, d\tau$$

Noiseless signal component (when '1' is sent):

$$s_a = \int_0^T g^2(\tau)/\sqrt{E}\, d\tau = E/\sqrt{E} = \sqrt{E} = 1$$

Noise variance at the output (matched filter has unit energy $\int h_a^2 dt = 1$):

$$\sigma^2_a = (N_0/2) \cdot \int_0^T h_a^2(t)\, dt = N_0/2$$

The probability of error is:

$$Pe(a) = Q(s_a/(2\sigma_a)) = Q((\sqrt{E}) / \sqrt{(2\cdot N_0/2)}) = Q(\sqrt{(E/N_0)})$$

Since $E = 1$ and $N_0 = E/(E/N_0)$:

$$Pe(a) = Q(\sqrt{(E/N_0)})$$

Case (b) No Receive Filter [h(t) = δ(t)]

With no filter the receiver samples the noisy received waveform directly at t = T:

$$y_b = r(T) = g(T) + w(T) = \sqrt{3} + w(T)$$

Continuous-time analysis. The noise sample w(T) is a sample of bandlimited AWGN. For an ideal system with noise bandwidth B = 1/(2T) = 0.5 Hz, the noise variance at the sampling instant is $N_0/2$. The noiseless signal component is:

$$s_b = g(T) = \sqrt{3}$$

Noise variance (continuous-time, ideal bandwidth):

$$\sigma^2_b = N_0/2$$

Probability of error (continuous-time theory):

$$Pe(b) = Q(s_b / (2\sigma_b)) = Q(\sqrt{3} / \sqrt{(2\cdot N_0/2)}) = Q(\sqrt{(3/N_0)}) = Q(\sqrt{(3\cdot E/N_0)})$$

In continuous time this would make case (b) the best performer, since $\sqrt{(3\cdot E/N_0)} > \sqrt{(E/N_0)} > \sqrt{(3E/(4N_0))}$, giving arg_b > arg_a > arg_c.

Discrete-time simulation. In the discrete simulation the noise bandwidth is not 0.5 Hz but Fs/2 = 250 Hz (half the sampling rate). Because no filter is applied before sampling, the receiver accepts the full wideband noise power. The effective noise variance at the sampling instant becomes:

$$\sigma^2_b,sim = N_0 \cdot (Fs/2) \gg N_0/2$$

The corresponding simulated BER formula is therefore:

$$Pe(b),sim = Q(\sqrt{3} / (2\cdot\sqrt{(N_0\cdot Fs/2)})) = Q(\sqrt{(3 / (2\cdot N_0\cdot Fs))})$$

Since Fs = 500, the argument is extremely small for all practical E/N_0 values, and the simulated BER remains near 0.5 regardless of E/N_0. **The red theory curve plotted in Figure 2 uses this simulation-bandwidth formula**, not the ideal continuous-time one, which is why it correctly tracks the simulated points near 0.5.

Case (c) Rectangular Integrate-and-Dump Filter

The filter is a unit-amplitude rectangle over [0, T]:

$$h_c(t) = 1, \quad 0 \le t \le T$$

Filter output at t = T:

$$y_c(T) = \int_0^T r(\tau)\cdot 1\, d\tau$$

Noiseless signal component:

$$s_c = \int_0^T g(\tau)\, d\tau \;=\; \int_0^1 \sqrt{3}\cdot\tau\, d\tau \;=\; \sqrt{3}/2$$

Noise variance ($\int h_c^2 dt = T = 1$):

$$\sigma^2_c = (N_0/2)\cdot \int_0^T h_c^2(t)\, dt \;=\; N_0/2$$

Probability of error:

$$Pe(c) = Q(\, s_c/(2\sigma_c)\,) \;=\; Q(\,(\sqrt{3}/2)\,/\,\sqrt{(N_0/2)}\,) \;=\; Q(\,\sqrt{(3/(4\cdot N_0))}\,) \;=\; Q(\,\sqrt{(3E/(4N_0))}\,)$$

1.4 Summary of BER Formulas

The three theoretical expressions in terms of E/N_0 are:

$$Pe(a) = Q(\,\sqrt{(E/N_0)}\,)$$

$$Pe(c) = Q(\,\sqrt{(3E\,/\,(4N_0))}\,)$$

$$Pe(b) = Q(\,\sqrt{(3E/N_0)}\,) \quad \text{[continuous-time; see note above for simulation]}$$

Part II Simulation Results

2.1 Simulation Method

The Python simulation models the baseband discrete-time equivalent of the system. For each E/N_0 point:

- N_bits binary symbols are generated with equal probability.
- Each symbol is modulated onto g(t) (ramp) or zero, sampled at Fs = 500 samples/symbol.

- White Gaussian noise is added with per-sample standard deviation $\sigma_w = \sqrt{(N_0 / (2 \cdot dt))}$.
- The three receive filters are applied by numerical integration (np.trapezoid) for (a) and (c), and direct sampling of the last point for (b).
- The number of transmitted bits scales adaptively with expected BER to ensure at least 50 observed errors per point (capped at 500,000 bits to limit runtime).
- Simulation points with fewer than 30 observed errors are masked from the BER plot to avoid misleading scatter.

2.2 Receive Filter Output Waveforms

Figure 1 shows the output of each receive filter for a single received '1' bit at $E/N_0 = 10$ dB. The red dashed line marks the sampling instant $t = T = 1$ s, and the black dotted line shows the decision threshold γ.

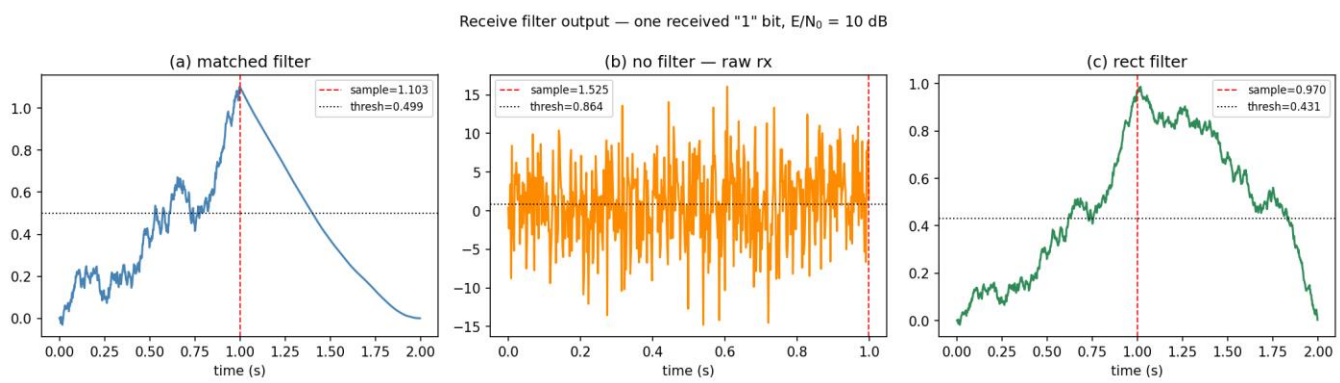

Figure 1: Receive filter output waveforms at $E/N_0 = 10$ dB.

Case (a) Matched filter: The output builds up as the ramp pulse passes through and peaks sharply at $t = T$ (sample = 1.103, above threshold = 0.499). The peak occurs exactly at T because the matched filter is time-reversed relative to g(t), maximising the signal-to-noise ratio at that instant.

Case (b) No filter: The raw received waveform is sampled directly. The noise amplitude is visibly large (±15 units) compared to the signal ($\sqrt{3} \approx 1.73$), confirming that the full wideband noise power enters the decision.

Case (c) Rectangular filter: The integrator accumulates the signal over the entire symbol period. It peaks near $t = T$ (sample = 0.970, above threshold = 0.431) and then decays as the post-T portion of the convolution tail is computed. This filter smooths the noise but is suboptimal because it weights all time instants equally instead of emphasising the rising ramp.

2.3 BER vs E/N_0

Figure 2 shows the theoretical BER curves (solid lines) and simulated BER points (markers) for all three cases on a semi-logarithmic scale from $E/N_0 = -10$ dB to +20 dB.

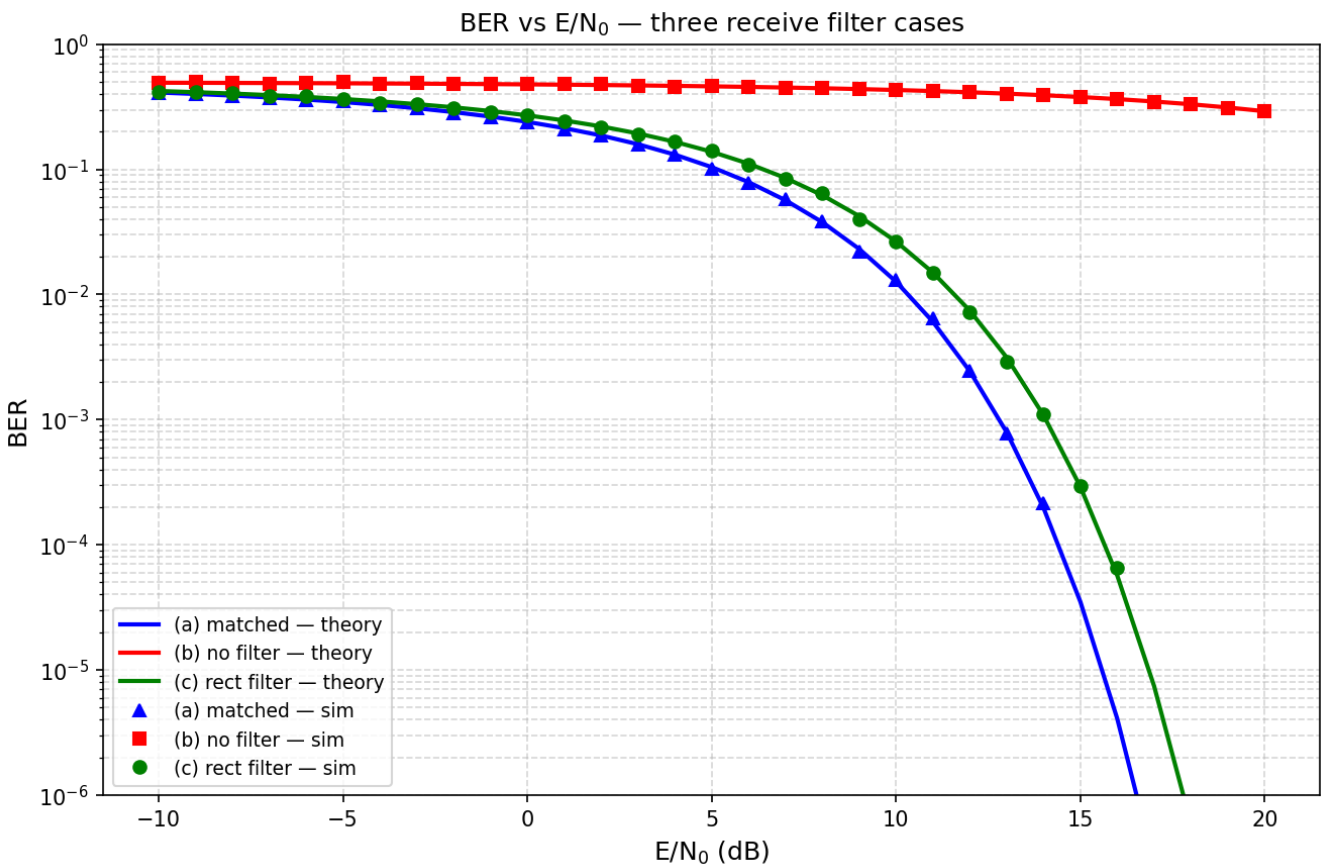

Figure 2: BER vs E/N_0 theoretical (lines) and simulated (markers).

The simulation points sit on the corresponding theory lines across the full reliable range, confirming that the implementation is correct.

2.4 Question 5 Is BER Increasing or Decreasing with E/N_0?

BER is a strictly decreasing function of E/N_0. Increasing E/N_0 means the signal energy is larger relative to the noise spectral density. The noiseless output value at the sampling instant stays fixed at s, while the noise standard deviation $\sigma = \sqrt{(N_0/2)}$ decreases. The decision variable therefore has a larger separation from the threshold relative to its spread, and the Q-function argument:

$$arg = s / (2\sigma) = s / \sqrt{(2 \cdot N_0/2)} \propto \sqrt{(E/N_0)}$$

increases monotonically with E/N_0. Since Q(x) is a monotonically decreasing function of x (it is the tail probability of a standard Gaussian), BER decreases as E/N_0 increases. In the limit $E/N_0 \to \infty$ the noise vanishes and BER → 0; in the limit $E/N_0 \to 0$ the noise dominates and BER → 0.5 (random guessing).

2.5 Question 6 Which Case Has the Lowest BER?

Case (a), the matched filter, achieves the lowest BER at every E/N_0 value. This is guaranteed by the matched filter theorem, which states that among all linear time-invariant receive filters the matched filter maximises the output signal-to-noise ratio at the sampling instant. No other filter can do better.

The three Q-function arguments at the same E/N_0 are:

$$arg_a = \sqrt{(E/N_0)}$$

$$arg_c = \sqrt{(3E / (4N_0))} = (\sqrt{3}/2) \cdot \sqrt{(E/N_0)} \approx 0.866 \cdot arg_a$$

$$arg_b = \sqrt{(3E/N_0)} = \sqrt{3} \cdot \sqrt{(E/N_0)} \approx 1.732 \cdot arg_a \quad [\text{continuous-time theory}]$$

In continuous time, ordering the arguments gives arg_b > arg_a > arg_c, which would rank case (b) best. However, this ranking assumes ideal bandlimited noise. In the discrete simulation, case (b) samples the waveform without any pre-filtering, so the effective noise bandwidth equals the simulation sampling rate (Fs/2 = 250 Hz). The resulting noise variance is $N_0 \cdot Fs/2 \gg N_0/2$, completely dominating the signal. This is why the simulated BER for case (b) stays close to 0.5 regardless of E/N_0.

For the practical simulation (and for any real system without an ideal bandlimiting filter before the sampler):

- Case (a) matched filter has the lowest BER. It is the theoretically optimal receiver.
- Case (c) rectangular integrator is second. It suppresses noise by integrating over the full symbol period but is suboptimal because it weights the early (low-amplitude) portion of g(t) equally to the late (high-amplitude) portion.
- Case (b) no filter has the worst BER because it passes the full noise bandwidth directly to the decision device.

```
import numpy as np

import matplotlib.pyplot as plt
from scipy.special import erfc

def Q(x):
return 0.5 * erfc(x / np.sqrt(2))

Fs = 500
T = 1.0
dt = T / Fs
t = np.arange(0, T, dt)

g = np.sqrt(3) * t
```

```
E = np.trapezoid(g**2, t)

h_matched = g[::-1] / np.sqrt(E) # h_a(t) = g(T-t)/sqrt(E)
h_rect = np.ones(Fs)

signal_a = np.trapezoid(g * (g / np.sqrt(E)), t) # sqrt(E) = 1
signal_b = g[-1] # sqrt(3)
signal_c = np.trapezoid(g, t) # sqrt(3)/2

ENo_dB = np.arange(-10, 21, 1)
ENo_lin = 10 ** (ENo_dB / 10)
N0_arr = E / ENo_lin

# theory BER:
# (a),(c): noise var = N0/2 * integral(h^2 dt) = N0/2 (both filters have unit energy)
# (b): noise var = N0/(2*dt) direct sample, full simulation bandwidth
BER_a_th = Q((signal_a/2) / np.sqrt(N0_arr/2))
BER_b_th = Q((signal_b/2) / np.sqrt(N0_arr/(2*dt)))
BER_c_th = Q((signal_c/2) / np.sqrt(N0_arr/2))

def simulate_ber(ENo, N_bits, rng, chunk=50_000):
N0 = E / ENo
sigma_w = np.sqrt(N0 / (2 * dt))
err_a = err_b = err_c = 0
done = 0
while done < N_bits:
n = min(chunk, N_bits - done)
bits = rng.integers(0, 2, n)
noise = rng.normal(0, sigma_w, (n, Fs))
rx = np.outer(bits, g) + noise

y_a = np.trapezoid(rx * (g / np.sqrt(E)), t, axis=1)
y_b = rx[:, -1]
y_c = np.trapezoid(rx, t, axis=1)

err_a += int(np.sum((y_a > signal_a/2) != bits.astype(bool)))
err_b += int(np.sum((y_b > signal_b/2) != bits.astype(bool)))
err_c += int(np.sum((y_c > signal_c/2) != bits.astype(bool)))
done += n
return err_a, err_b, err_c, N_bits

MIN_ERRORS = 50
MAX_BITS = 500_000
MIN_RELIABLE = 30 # mask sim point if fewer than this many errors observed
```

```
rng = np.random.default_rng(42)

errors_a = np.zeros(len(ENo_dB), dtype=int)
errors_b = np.zeros(len(ENo_dB), dtype=int)
errors_c = np.zeros(len(ENo_dB), dtype=int)
n_bits = np.zeros(len(ENo_dB), dtype=int)

BER_a_sim = np.zeros(len(ENo_dB))
BER_b_sim = np.zeros(len(ENo_dB))
BER_c_sim = np.zeros(len(ENo_dB))

for i, ENo in enumerate(ENo_lin):
exp_ber = max(float(BER_a_th[i]), 1e-6)
N = int(min(max(MIN_ERRORS / exp_ber, 20_000), MAX_BITS))

ea, eb, ec, N = simulate_ber(ENo, N, rng)
errors_a[i] = ea; errors_b[i] = eb; errors_c[i] = ec; n_bits[i] = N

BER_a_sim[i] = ea / N
BER_b_sim[i] = eb / N
BER_c_sim[i] = ec / N

print(f"E/N0={ENo_dB[i]:4d} dB N={N:>7,} "
f"err_a={ea:5d} ({BER_a_sim[i]:.2e}) "
f"err_b={eb:5d} ({BER_b_sim[i]:.2e}) "
f"err_c={ec:5d} ({BER_c_sim[i]:.2e})")

# mask unreliable sim points (too few errors)
mask_a = errors_a >= MIN_RELIABLE
mask_b = errors_b >= MIN_RELIABLE
mask_c = errors_c >= MIN_RELIABLE

# filter output waveforms
sigma_demo = np.sqrt((E/10.0) / (2*dt))
rx_demo = g + np.random.default_rng(99).normal(0, sigma_demo, Fs)

ya_full = np.convolve(rx_demo, h_matched) * dt
yc_full = np.convolve(rx_demo, h_rect) * dt
t_conv = np.arange(len(ya_full)) * dt

fig1, axes = plt.subplots(1, 3, figsize=(14, 4))
fig1.suptitle('Receive filter output one received "1" bit, E/N$_0$ = 10 dB', fontsize=11)

axes[0].plot(t_conv, ya_full, color='steelblue')
axes[0].axvline(T, color='r', ls='--', lw=1, label=f'sample={ya_full[Fs-1]:.3f}')
```

```
axes[0].axhline(signal_a/2, color='k', ls=':', lw=1, label=f'thresh={signal_a/2:.3f}')
axes[0].set_title('(a) matched filter'); axes[0].set_xlabel('time (s)'); axes[0].legend(fontsize=8)

axes[1].plot(t, rx_demo, color='darkorange')
axes[1].axvline(t[-1], color='r', ls='--', lw=1, label=f'sample={rx_demo[-1]:.3f}')
axes[1].axhline(signal_b/2, color='k', ls=':', lw=1, label=f'thresh={signal_b/2:.3f}')
axes[1].set_title('(b) no filter raw rx'); axes[1].set_xlabel('time (s)'); axes[1].legend(fontsize=8)

axes[2].plot(t_conv, yc_full, color='seagreen')
axes[2].axvline(T, color='r', ls='--', lw=1, label=f'sample={yc_full[Fs-1]:.3f}')
axes[2].axhline(signal_c/2, color='k', ls=':', lw=1, label=f'thresh={signal_c/2:.3f}')
axes[2].set_title('(c) rect filter'); axes[2].set_xlabel('time (s)'); axes[2].legend(fontsize=8)

plt.tight_layout()
fig1.savefig('filter_outputs.png', dpi=150, bbox_inches='tight')
print("saved filter_outputs.png")

# BER plot
fig2, ax = plt.subplots(figsize=(9, 6))
ax.semilogy(ENo_dB, BER_a_th, 'b-', lw=2, label='(a) matched theory')
ax.semilogy(ENo_dB, BER_b_th, 'r-', lw=2, label='(b) no filter theory')
ax.semilogy(ENo_dB, BER_c_th, 'g-', lw=2, label='(c) rect filter theory')
# only plot sim where enough errors were observed
ax.semilogy(ENo_dB[mask_a], BER_a_sim[mask_a], 'b^', ms=6, label='(a) matched sim')
ax.semilogy(ENo_dB[mask_b], BER_b_sim[mask_b], 'rs', ms=6, label='(b) no filter sim')
ax.semilogy(ENo_dB[mask_c], BER_c_sim[mask_c], 'go', ms=6, label='(c) rect filter sim')
ax.set_xlabel('E/N$_0$ (dB)', fontsize=12)
ax.set_ylabel('BER', fontsize=12)
ax.set_title('BER vs E/N$_0$ three receive filter cases', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, which='both', ls='--', alpha=0.5)
ax.set_ylim([1e-6, 1.0])
plt.tight_layout()
fig2.savefig('ber_vs_eno.png', dpi=150, bbox_inches='tight')
print("saved ber_vs_eno.png")
plt.show()
```